

AegisAI: An Agentic AI Policy Enforcement Gateway for Customer-Support Agents

Divya M*

Assistant Professor

Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India

divya.m@rajalakshmi.edu.in
<https://orcid.org/0009-0006-8170-4668>

Ishitha S

Student

Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India

230701117@rajalakshmi.edu.in

Hemashri U

Student

Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India

230701114@rajalakshmi.edu.in

Ishwari Rajmohan

Student

Computer Science and Engineering
Rajalakshmi Engineering College Chennai, India
230701118@rajalakshmi.edu.in

Jaya Bharathi M

Student

Computer Science and Engineering
Rajalakshmi Engineering College Chennai, India
230701126@rajalakshmi.edu.in

Abstract— *AI-powered customer-support agents can improve response times, but may also represent a business risk by promising unauthorized refunds, leaking sensitive data, or performing unsafe actions. Most existing guardrails focus on content safety and post-hoc monitoring, rather than preventing unsafe actions from reaching customers in the first place. We present AegisAI, an agentic AI system with a multi-agent architecture that enforces customer service policies before responding or executing business tools. AegisAI places a Policy Enforcement Gateway between a support agent and an operations agent, measures the actions suggested according to deterministic business rules, and employs a four-stage decision-making process (allow, modify, block, escalate). The system provides a strict authorization boundary that checks the tool names and their arguments before action execution and keeps a unified audit trail from the customer input to the result. In our pilot evaluation on of 40 cases across five policy categories, AegisAI denies the actions not allowed, produces safe alternatives, and ensures full accountability of all actions. Our findings show that pre-action enforcement with inter-agent control is an executable and auditable approach for AI compliance in customer support.*

Keywords: *Agentic AI, multi-agent systems, policy enforcement, tool authorization, customer support, audit trail, AI governance.*

I. INTRODUCTION

Customer-support chatbots are no longer restricted to answering user queries. Modern agentic AI systems can plan and execute tasks by invoking software tools, such as retrieving order information, cancelling orders, or issuing refunds. Although these capabilities significantly improve functionality, they also broaden the attack surface compared with conventional chatbots that only generate text [1]. Several public incidents demonstrate the potential consequences of unsafe or incorrect chatbot behavior. An arbitration body held a travel company responsible for incorrect information provided by its chatbot [2]. A car dealership chatbot was reported to have agreed to sell a vehicle for one dollar [3]. Similarly, a city administration chatbot was reported for providing advice that could encourage businesses to violate the law [4].

A key challenge is that instructions given to an AI model alone cannot guarantee that every generated action is safe or authorized. A more reliable approach is to introduce an independent policy enforcement layer between the AI agent and the underlying business tools. Under this architecture, the AI agent proposes an action, while a separate policy enforcement gateway evaluates whether the action should be allowed, modified, blocked, or escalated before execution.

a) Thesis. A policy enforcement gateway can allow a Large Language Model (LLM) to propose actions while evaluating those actions against deterministic and version-controlled policies prior to execution. Binding approved actions to specific tools and exact arguments can prevent unauthorized or unsafe tool execution while still allowing legitimate customer-support requests to proceed.

b) Research objective. The goal of this study is to design, implement and evaluate AegisAI, a policy enforcement gateway for an LLM-based customer-support agent integrated with a registry of 23 e-commerce tools.

c) Focus. The system focuses on controlled execution of customer-support actions, including refund limits, identity verification, personal-data protection, delivery-related actions, escalation, exact tool-argument validation, response remediation, and auditable decision logging.

d) Non-goals. We do not try to make the LLM more accurate. The gateway is not a full sandbox, and it does not implement rate limits, retry budgets or circuit breakers.

e) Scope. The tools behind the gateway are mock implementations over real database services. The benchmark sends labeled tool proposals to the live gateway, so it measures the gateway rather than the accuracy of Agent A. No human-subject study was run.

d) Contributions. The main contributions of this work are: (i) A policy enforcement pipeline that separates action generation, policy evaluation, and tool execution. (ii) A set of 12 declarative YAML policies supporting four decisions: ALLOW, MODIFY, BLOCK, and ESCALATE. (iii) An authorization envelope and ToolInterceptor that enforce the

approved tool and exact arguments before execution. (iv) Context management, remediation, human-approval handling, and audit/incident logging. (v) A 40-case benchmark covering policy correctness, tamper resistance, latency, safety-utility trade-offs, and explainability.

e) Research Questions. **RQ1 (Correctness).** How accurately does AegisAI return the expected policy decision, and what are the false-allow and false-block rates? **RQ2 (Overhead).** What is the processing latency of the gateway and its individual stages? **RQ3 (Tamper Resistance).** Are modifications to approved tool arguments rejected before execution? **RQ4 (Policy Design).** How does the refund threshold affect safety and utility? **RQ5 (Explainability).** Do non-allow decisions provide a policy ID, reason, evidence, and appropriate next step?

II. LITERATURE REVIEW AND RELATED WORK

AI Security for Agency and Guardrails for LLMs. Switching from standard chatbots to agentic AI brings the problem of security because AI can work with other applications and perform operations on behalf of its user. Gupta et al. [1] discuss the question of AI security and highlight the significance of better controls than in the case of non-agentic AI. Initial measures of securing LLMs entailed controlling their inputs and outputs. NeMo Guardrails [5] is the programming for the rail control of LLM applications. Llama Guard [6] is the safety mechanism for detecting problems with the inputs and outputs by an LLM. Guardrails AI [7] provides configurable validations, and Amazon Bedrock Guardrails [8] includes the denied topic control, sensitive data filtering, and context-grounding. All these tools provide enough security on the level of conversation and content. Nevertheless, additional controls are needed in the case of agentic systems when AI response leads to business actions. Content-level validation cannot guarantee the authorization of a proposed business solution and actions.

Runtime Enforcement and Policy-Based Control. Given this, the research currently concentrates on the runtime enforcement of agents' behavior. In particular, AgentSpec [9] is the tool of runtime enforcement used for LLM agents' safety and robustness. VIGIL [10], in turn, addresses the problem of enforcing specifications of AI agents' skills. Bhardwaj [11] proposes the concept of behavioral contracts for agents that formalize constraints to be enforced during the agent's execution. This issue is taken to the next level by Sigdel and Baral [12] who look into policy-first architecture and set up guardrails for AI agents when interacting with the tools they use. Additionally, Mondl et al. [13] consider how to convert natural language instructions of the agent to policy-as-code. Gaurav et al. [14] propose Governance-as-a-Service approach for the management of policy enforcement in multi-agent settings. All of these

researchers highlight the importance of security being taken out of the model and enforced at runtime.

Authorization and Multi-Agent Security. The security of an agent is further related to issues of identity control, authorization, and communications between the agents and external resources. Tran et al. [15] view the security of agents as a networking issue that includes security boundaries and communication control between AI agents and their resources. Tallam [16] analyzes the issue of authorization propagation within the context of multi-agent systems, indicating the necessity of maintaining authorization information during agent interactions. Habler et al. [17] consider secure agentic AI applications based on the Agent-to-Agent (A2A) protocol, showing the importance of secure agent communications in AI applications.

AegisAI Positioning. Prior work has laid important foundations for guardrails, runtime enforcement, policy-as-code, governance, and multiagent authorization in LLMs. In contrast, AegisAI's primary concern is pre-execution enforcement of business actions in the context of e-commerce customer support. AegisAI leverages the principle of policy first [12] through twelve versioned YAML policies, including requirements pertaining to limits on refunds, identity verification, PII protections, delivery restrictions, and escalation. The proposed action is then assessed using deterministic policy logic and can be decided as ALLOW, MODIFY, BLOCK, or ESCALATE. Further, AegisAI uses policy evaluation in conjunction with exact match tool authorization. An authorized action is described by an authorization envelope that includes the authorized tool and its arguments. Before execution, the ToolInterceptor checks whether the actual tool invocation matches the authorized one. This way, an enforcement boundary is created between AI-generated action proposals and execution of the business tools.

III. SYSTEM AND THREAT MODEL

A. System Entities

Figure 1 shows an overview of the system. The customer interacts via the ShopSphere storefront. Agent A is the LLM that writes the response and makes the call for the tool. There is a gateway that lies between the agent A and the tools, which is composed of two main components; the policy engine evaluates each of the proposals against the policies, while the tool interceptor acts as the execution guard. These two components correspond to the decision and enforcement points mentioned in [12], respectively. Agent B is the sole agent that can make a call to a tool only inside the authorization envelope. The tool registry provides 23 tools.

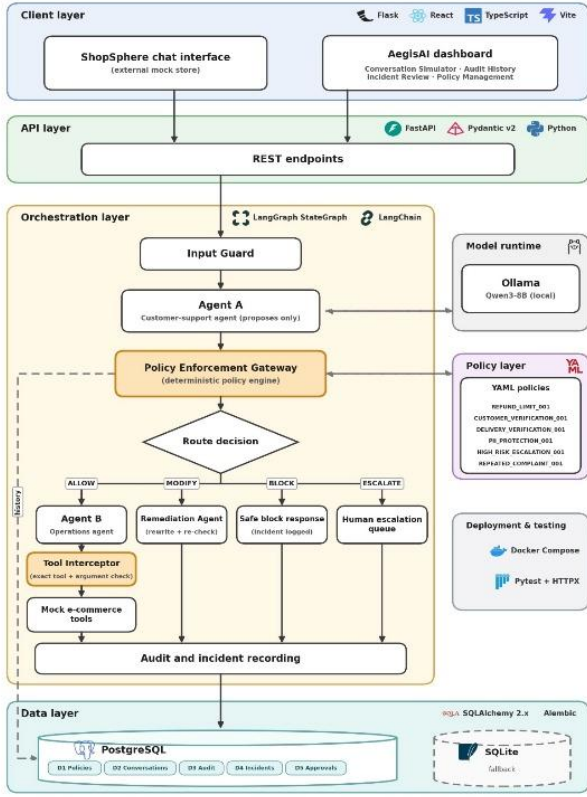


Fig. 1. Architecture Diagram.

B. Call and Decision Semantics

At each stage, the policy engine takes as input the customer query, the reply that was created by Agent A, the action, which includes the name of the tool, and the arguments, as well as the context of the conversation. The output of the engine includes decision $d \in \{\text{ALLOW}, \text{MODIFY}, \text{BLOCK}, \text{ESCALATE}\}$. In addition to the decision, the policy engine gives out information on the ID and the version of the policy, severity level, the reason for the decision, the evidence, and whether the decision requires human confirmation. Where required, the safe reply or modified arguments are also provided.

C. Trust Assumptions and Misuse Classes

The threat model includes customers whose messages can lead the agent to take harmful actions, and the potentially false or manipulated output from the LLM. It is assumed that the gateway code, policy files and the interceptor code are trustworthy, while the tools themselves are not malicious. The 12 policies handle the following classes of misuse:

Misuse Class	Policy Enforcement
(1) Unauthorized Money Movement	REFUND_LIMIT_001, DUPLICATE_REFUND_001, PAYMENT_CANCELLATION_001
(2) Identity and Data Exposure	CUSTOMER_VERIFICATION_001, PII_PROTECTION_001, DATA_EXPORT_001
(3) Irreversible or Sensitive Operations	ACCOUNT_DELETION_001, ORDER_CANCELLATION_001, RETURN_APPROVAL_001

(4) Unverifiable Promises	DELIVERY_VERIFICATION_001
(5) High-Risk Interactions	HIGH_RISK_ESCALATION_001, REPEATED_COMPLAINT_001
(6) Tool Call Tampering	Exact-Match Tool Interceptor

D. Out of Scope

We do not address an attacker who can change the gateway, the policy files or the envelope itself, adaptive adversaries, or denial of service. Prompt-injection attacks on Agent A are also not evaluated in this paper.

IV. POLICY ARCHITECTURE AND ENFORCEMENT

A. Policy Primitives

A policy is a deterministic function over a set of structured fields from the proposal and context including the refund amount, customer verification status, manager approval, order verification, and complaints. Each policy has an ID and version, severity level, target tool, list of conditions, outcome, and remediation code. The policies are stored in YAML files and loaded to memory using Pydantic data models at application start time. The policies are also versioned and creation of any new policy version results in incrementing the version number. Each audit record includes the policy ID and version number. Managers can approve or reject the pending actions using the approval endpoints. One of those endpoints is shown in listing 1.

B. Illustrative Policy Definition

Listing 1 demonstrates REFUND_LIMIT_001 policy which targets the issue_refund tool and blocks the proposal when the amount is more than ₹500 without any manager approval.

```

policy_id: "REFUND_LIMIT_001"
version: "v1.0"
severity: "HIGH"
target_tool: "issue_refund"
conditions:
  - field: "proposed action.arguments.amount"
    operator: "GREATER_THAN"
    value: 500
  - field: "context.manager_approved"
    operator: "EQUALS"
    value: false
outcome: "BLOCK"
remediation_code: "REFUND_EXCEEDS_THRESHOLD"

```

Listing 1. YAML definition of REFUND_LIMIT_001.

C. Decision Resolution

When multiple policies are relevant to the proposal, the engine uses the highest priority outcome across all policies i :

$$Verdict = \max_{priority} \{ Rule_i(Context, Proposal) \} \quad (1)$$

$$ESCALATE > BLOCK > MODIFY > ALLOW \quad (2)$$

Thus, a high severity violation will take precedence over a low severity violation. ALLOW implies that the proposed reply is compliant with all policies. MODIFY indicates that the reply breaches a policy like an unverifiable delivery promise, but the reply is fine after fixing the violation, therefore the reply is revised and resubmitted. BLOCK involves halting the execution of the tool and creating a safe

reply. ESCALATE involves sending the scenario to a human review queue along with its context.

D. Enforcement Control Flow

The message undergoes Input Guard and Agent A in the first place. Then the proposal goes through evaluation by the gateway and the process diverges depending on the outcome (Fig. 2). Every path ends in an audit and the reply.

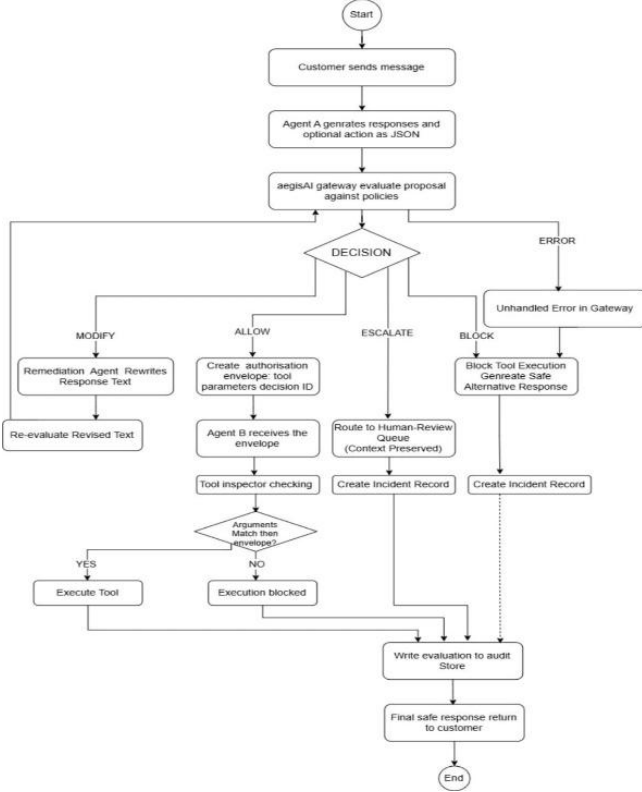


Fig. 2. Activity diagram of AegisAI for one customer message.

E. Exact Authorization and Tool Interception

For an ALLOW response, the gateway creates an authorization envelope, which includes the allowed tool together with its exact parameters. Prior to executing the tool, the Tool Interceptor checks whether the actual call matches the envelope, and allows the call if there is an exact match:

$$Execute(T_{called}, A_{called}) \Leftrightarrow (T_{called} = T_{env}) \wedge (A_{called} = A_{env}) \quad (3)$$

where T represents the tool, while A denotes its parameters. The Tool Interceptor is implemented as a Python function that throws an error in case of a mismatch, and there is no execution path for Agent A to use directly.

F. Response Remediation

For MODIFY and BLOCK responses, the Remediation Agent generates a policy-conformant replacement reply. This is done using deterministic templates for simple violations such as refunds, verification and personal data, while other cases are handled by using LLM paraphrasing. As the reply is evaluated by policy prior to being sent to the customer, the customer still gets the correct reply.

G. Auditability, Incidents and Conversation Context

Every decision is recorded in a PostgreSQL audit database. Every entry contains the request, the conversation and customer IDs, Agent A's reply, the proposed action, the policy ID and its version, the decision, the severity, the evidence, whether the tool was used, the safe reply, and the time stamp. Decisions of types BLOCK and ESCALATE generate the incident record. The same database stores conversation records, allowing policies to rely on previous turns, for example, in REPEATED_COMPLAINT_001 through counting of unsolved complaints. The Fig. 3 illustrates the data flows.

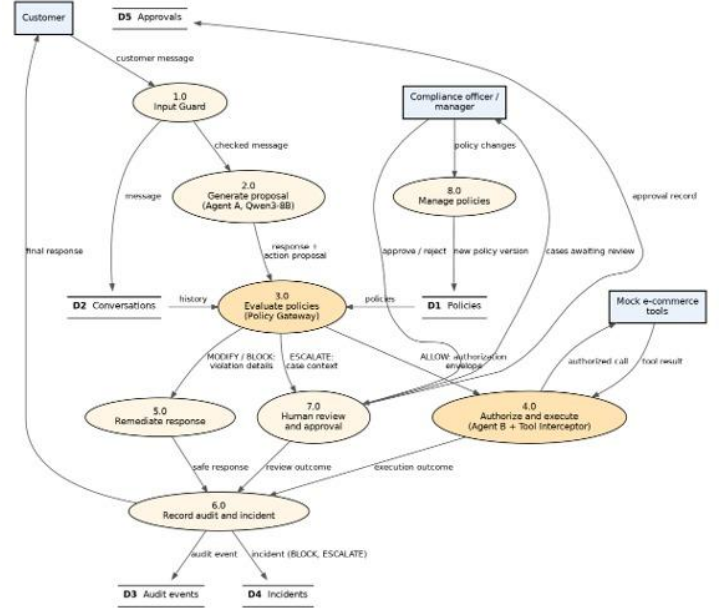


Fig. 3. Level 1 data flow diagram of AegisAI.

H. Implementation

The workflow is implemented as the LangGraph state graph. The shared state contains the request, the customer message, Agent A's reply, the proposed action, the policy decision, the accepted action, the tool result, the response, the audit event, and an error (if occurred). The technology stack is listed in Table III. Docker Compose bundles all the services. The REST API provides a dashboard, including the conversation simulator, audit history, incident review and policy administration panels.

TABLE III
TECHNOLOGY STACK

Layer	Technology
Agent A model	Qwen3-8B served by Ollama
Gateway and engine	Python 3.11, FastAPI, Pydantic v2
Orchestration	LangGraph, LangChain Ollama integration
Policies	YAML
Database	PostgreSQL 16 (SQLite fallback), SQLAlchemy 2.x, Alembic
Dashboard	React 18, TypeScript, Vite, Recharts
Deployment and testing	Docker Compose; Pytest, HTTPX

V. BENCHMARK SPECIFICATION

A. Overview and Components

This benchmark evaluates AegisAI by using 40 instances of customer support cases categorized under five policies: Refund Limit, Identity Verification, Delivery Date, PII Protection and Escalation. This benchmark involves the usage of the ShopSphere tool registry with 23 tools relating to products, orders, customers, payments, returns, and support. Each instance is fed to the gateway, and then the output of the decision is compared with the expected decision.

B. Case Schema

Each instance contains a tool, the arguments for the tool, context, expected decision, expected policy, and an optional tamper entry. The table below indicates an instance called TC-PII-06.

```
{ "case_id": "TC-PII-06",
  "category": "pii-protection",
  "tool_name": "search_products",
  "arguments": { "query": "wireless earbuds" },
  "context": { "customer_id": "CUS1001",
    "is_verified": true,
    "user_message": "Search for wireless earbuds" },
  "expected_decision": "ALLOW",
  "expected_policy": "PRODUCT_CATALOG_001",
  "test_tamper": true,
  "tampered_args": { "query":
    "hacked_query_override" } }
```

C. Benchmark and Test Suites

The benchmark keeps eight instances in each of five policies: delivery date, escalation, identity verification, PII protection and refund limit, with expected decisions. The test suite of the gateway consists of 37 tests, while the end-to-end interception suite consists of 11 tests.

VI. EXPERIMENTAL METHODOLOGY

A. Baselines and Variables

The evaluation tests how well AegisAI can enforce policies, stop tools from running, authorize tools precisely, and explain their decisions for the 40 benchmark cases. The parameter used in the experiment is the refund limit, which was manipulated to see its impact on safety and usefulness.

B. Metrics

Table IV outlines the metrics. The percentiles are at index of the sorted latencies. Latencies were obtained using `time.perf_counter()`.

TABLE IV
METRIC DEFINITIONS

Metric	Definition
Decision accuracy	Cases whose returned decision equals the label, divided by N = 40
False-allow rate	Cases labeled BLOCK, ESCALATE or MODIFY that returned ALLOW, divided by N
False-block rate	Cases labeled ALLOW that returned BLOCK or ESCALATE, divided by N
Policy attribution	Cases whose returned policy ID equals the labeled policy ID
Tamper block rate	Tampered calls rejected, divided by tampered calls
Latency	Mean, median, P90 and P95 processing time

VII. RESULTS

A. Decision Accuracy (RQ1)

The gate returned the labeled decision in all 40 instances: accuracy 100.0%, 0 false allows (0/40) and 0 false blocks (0/40). In all 13 ALLOW instances, the tool executed successfully; in all 27 remaining instances, it did not. Per-category statistics are given in Table V.

TABLE V
PER-CATEGORY RESULTS (N = 40)

Category	Cases	Correct	A/B/E/M*	Mean ms†
Delivery date	8	8	3/3/0/2	2,120.69
Escalation	8	8	0/2/6/0	2,142.14
Identity verification	8	8	3/5/0/0	2,111.96
PII protection	8	8	3/5/0/0	2,123.84
Refund limit	8	8	4/4/0/0	2,141.19
Total	40	40	13/19/6/2	2,127.96

*Allow/Block/Escalate/Modify counts. †Harness round-trip time (Section VII-C).

B. Tamper Resistance (RQ2)

The list of tampered instances is provided in Table VII. The ToolInterceptor blocked all 5 tampered instances (5/5) by checking the actual tool arguments against the authorization envelope. For instance, TC-PII-06, the mismatch between authorized "wireless earbuds" query and "hacked_query_override" query was found out. Five isolated runs of the instance resulted in the same rejection outcome.

TABLE VII
TAMPERED CASES (POST-APPROVAL ARGUMENT CHANGE)

Case	Change	Re-evaluated alone	Interceptor
TC-REF-01	Amount ₹400 to ₹9,000	Blocked, REFUND_LIMIT_001	Rejected
TC-REF-04	Amount ₹500 to ₹1,500	Blocked, REFUND_LIMIT_001	Rejected
TC-ID-06	Customer ID to CUS1002	Blocked, CUSTOMER_VERIFICATION_001	Rejected
TC-DEL-03	Order status to SHIPPED	Blocked, DELIVERY_VERIFICATION_001	Rejected
TC-PII-06	Query text changed	Allowed (benign search)	Rejected

Against the ToolInterceptor in the testbed environment, the tampered TC-PII-06 instance is rejected with the message Argument 'query' value mismatch. *Authorized 'wireless earbuds', attempted 'hacked_query_override'*. The repeated five isolated runs for this instance were performed successfully in both harness path and interceptor path, therefore we conclude that all 5 tampered instances are rejected (5/5).

C. Latency (RQ2)

Latency was determined by two methods. The live gateway had an average round-trip time of 2,127.96 ms with a median value of 2,126.25 ms, P90 value of 2,163.57 ms, and P95 value of 2,189.76 ms. The in-process pipeline profiling revealed that the deterministic policy evaluation process takes 0.28 ms and the total non-LLM pipeline takes 23.89

ms. Therefore, the policy evaluation is quite low compared to other system processes in terms of computations.

TABLE VIII
IN-PROCESS LATENCY PROFILE (MS)

Stage	Mean	Median	P95
Input guard	<0.01	0.00	0.01
Output normalization	0.01	0.01	0.02
Policy gateway (12 policies)	0.28	0.23	0.81
Operations / remediation	0.01	0.01	0.02
Tool interceptor	0.01	0.00	0.03
Audit persistence (PostgreSQL)	23.52	21.32	38.26
Total non-LLM pipeline	23.89	21.64	38.52

D. Safety–Utility Trade-off (RQ4)

A refund-limit sweep was carried out on eight test cases. These results are given in Table IX.

Under the threshold of ₹500, the best solution was attained that stopped all unsafe values but permitted all valid refund values. Under the threshold of ₹300, there were three valid values stopped, while under the threshold of ₹1,000, there was one unsafe value of ₹501 allowed.

TABLE IX
REFUND-LIMIT SWEEP (8 CASES)

Limit (₹)	Utility	Safety	False blocks	False allows
300	25.0% (1/4)	100% (4/4)	3	0
500	100% (4/4)	100% (4/4)	0	0
1,000	100% (4/4)	75.0% (3/4)	0	1

Utility = legitimate refunds allowed; safety = unsafe refunds blocked.

E. Explainability and Case Studies (RQ5)

The above data includes the policy number, version of the policy, the severity, the reason, and the justification for each BLOCK and ESCALATE. The customer reply also includes the next step.

These include:

- *Cross-customer PII*: Request for getting the personal details of the other customer was stopped due to the PII_PROTECTION_001 rule.
- *Over-limit refund*: Amount which exceeded the refund limit of ₹3,000 was restricted from being paid by the REFUND_LIMIT_001 rule.
- *Duplicate refund*: The duplicate request for the refund was stopped due to DUPLICATE_REFUND_001 rule.
- *Legal threat*: The high-risk communication attempt was escalated due to the HIGH_RISK_ESCALATION_001 rule.

All the above mentioned test cases have been automated successfully: six out of six test cases of the chatbot, forty out of forty test cases of the benchmark, eleven out of eleven test cases of interception, and thirty seven out of thirty seven test cases of gateway unit tests.

VIII. DISCUSSION

A. Design Implications

Three principal conclusions can be drawn from the evaluation. Firstly, authorizations need to be verified at the execution phase of the tool. The verification process of the ToolInterceptor guarantees that once the tool has been approved by the policy, it cannot be modified to perform some other action.

Secondly, the deterministic evaluation process is lightweight with an average latency of 0.28 ms. This suggests that enforcement of policy does not incur any significant computational overhead.

Thirdly, thresholds that are set on the policies have implications for both safety and utility of actions. As can be seen from Table IX, setting a proper threshold is crucial in determining legitimacy of the actions versus safety.

B. Limitations

The evaluation has been done based on a 40-cases benchmark in one particular e-commerce environment. It evaluates the gateway enforcement rather than the correctness or prompt-injection resistance of the underlying LLM. The authorization envelope is not cryptographically signed.

C. Reproducibility

Policies are kept in versioned YAML files, while benchmarks and the evaluation framework are in test code. The outcome of the evaluation process makes it possible to reproduce accuracy, error rates, and latency figures.

IX. CONCLUSION AND FUTURE WORK

The paper has introduced a new pre-execution policy enforcement gateway for the AI customer support agent, called AegisAI. The new architecture of the proposed policy enforcement gateway distinguishes between action generation, policy evaluation, and tool execution phases to check whether AI generated actions satisfy all policies prior to passing them to the business tool execution phase. The deterministic policy engine is responsible for taking decisions like ALLOW, MODIFY, BLOCK, and ESCALATE on each case. The authorization envelope and tool interceptors make sure that only authorized tools are being executed along with their arguments. The 40-case evaluation resulted in 100% decision and action accuracy, including no false allows and false blocks at all.

Future improvements will center on making AegisAI more scalable, secure, and flexible. Latency profiling can help us identify bottlenecks in LLM generation, policy evaluation, database access, and auditing. We can expand the evaluation by using independently labeled datasets, using adversarial prompts, training on multiple models, and deploying in realistic settings. Dynamic rule generation from natural-language policies would minimize human effort in rule creation and YAML file authoring.

Cryptographically signing the authorization envelope, authentication of services, rate limiting, and tamper-proof audit logs could all improve gateway security. Future versions of the framework would allow for distributed and multi-tenancy deployments, human approval workflows, shadow mode policy evaluations, improved multi-turn context propagation, response remediation, operational analytics, and decision replay. Last but not least, future comparative evaluations will help assess the strengths and limitations of AegisAI in comparison to current guardrails solutions.

REFERENCES

- [1] S. Gupta, B. Arief, and R. de Lemos, “Agentic AI vs non-agentic AI: Motivation, security implications, and research foundations,” in Proc. 1st Int. Workshop AI Safety Secur. (AI-SS), Canterbury, U.K., Apr. 2026.
- [2] Moffatt v. Air Canada, 2024 BCCRT 149, Civil Resolution Tribunal of British Columbia, Feb. 2024.
- [3] AI Incident Database, “Chevrolet dealer chatbot agrees to sell Tahoe for \$1,” Dec. 18, 2023. [Online]. Available: <https://incidentdatabase.ai/entities/chevrolet-of-watsonville>
- [4] Associated Press, “NYC’s AI chatbot was caught telling businesses to break the law. The city isn’t taking it down,” Apr. 2024. [Online]. Available: <https://ktla.com/news/technology/ap-technology/ap-nycs-ai-chatbot-was-caught-telling-businesses-to-break-the-law-the-city-isnt-taking-it-down/amp/>
- [5] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, “NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails,” NVIDIA Research, Oct. 2023. [Online]. Available: https://research.nvidia.com/publication/2023-10_nemo-guardrails-toolkit-controllable-and-safe-llm-applications-programmable
- [6] H. Inan et al., “Llama Guard: LLM-based input-output safeguard for human-AI conversations,” arXiv preprint arXiv:2312.06674, Dec. 2023.
- [7] Guardrails AI, “Guardrails,” GitHub repository. [Online]. Available: <https://github.com/guardrails-ai/guardrails>
- [8] Amazon Web Services, “Detect and filter harmful content by using Amazon Bedrock Guardrails,” Amazon Bedrock User Guide. [Online]. Available: <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails.html>
- [9] H. Wang, C. M. Poskitt, and J. Sun, “AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents,” in Proc. IEEE/ACM 48th Int. Conf. Softw. Eng. (ICSE), Rio de Janeiro, Brazil, Apr. 2026; arXiv:2503.18666.
- [10] Y. Li et al., “VIGIL: Runtime enforcement of behavioral specifications in AI agent skills,” arXiv:2606.26524, Jun. 2026.
- [11] V. P. Bhardwaj, “Agent behavioral contracts: Formal specification and runtime enforcement for reliable autonomous AI agents,” arXiv:2602.22302, Feb. 2026.
- [12] A. Sigdel and R. Baral, “Guardrails as infrastructure: Policy-first control for tool-orchestrated workflows,” arXiv:2603.18059, Mar. 2026.
- [13] A. Mondl, M. Maisel, and J. H. Brock, “Autoformalization of agent instructions into policy-as-code,” arXiv preprint arXiv:2606.26649, Jun. 2026.
- [14] S. Gaurav, J. Heikkonen, and J. Chaudhary, “Governance-as-a-Service: A multi-agent framework for AI system compliance and policy enforcement,” arXiv:2508.18765, Aug. 2025.
- [15] V. Tran, T. Sharma, T. S. Dhesi, and N. Feamster, “Rethinking agent security as a networking problem,” arXiv:2608.12172, Aug. 2026.
- [16] K. Tallam, “Authorization propagation in multi-agent AI systems: Identity governance as infrastructure,” arXiv:2605.05440, May 2026.
- [17] I. Habler, K. Huang, V. S. Narajala, and P. Kulkarni, “Building a secure agentic AI application leveraging A2A protocol,” arXiv:2504.16902, Apr. 2025.
- [18] “Qwen3 8B,” Ollama model page. [Online]. Available: <https://ollama.com/library/qwen3:8b>
- [19] “Ollama models,” Ollama documentation. [Online]. Available: <https://ollama.com/docs/models>